COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	09/15/26	**DATE SUBMITTED:**	09/22/26
NAME:	Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quehado

ACTIVITY NO. 8 **[ACTIVITY NAME] Sorting Algorithms Pt2**

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 8-1. Array of Values for Sort Algorithm Testing

Source Code

```cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include "SortingAlgo_P2.h"

void printArray(const std::vector<int>& arr) {
    for (int val : arr) std::cout << val << " ";
    std::cout << "\n";
}

int main() {
    srand( Seed: time(0));
    std::vector<int> baseArr( n: 10);
    for (int& num : baseArr) num = rand() % 100 + 1;

    // Array Generation
    std::cout << "array of values\n";
    std::cout << "generated: ";
    printArray(baseArr);
```

Console Output

```
array of values
generated: 23 68 23 31 95 60 7 24 62 88
```

Observation

- The code generates a random array of 10 numbers between 1 and 100 using a random seed then This gives us a raw unsorted dataset to test our algorithms on and by saving this base array, we can put it too the 3 sorts below, using the exact same numbers into every sorting method to keep our test fair and consistent.

Table 8-2. Shell Sort Technique

Source Code

```
// Shell Sort
std::vector<int> arr  = baseArr;
std::cout << "\nshell sort\n";
std::cout << "unsorted: ";
printArray(arr);
shellSort( [&] arr);
std::cout << "sorted: ";
printArray(arr);
```

```
// Shell Sort
void shellSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i], j = i;
            while (j >= gap && arr[j - gap] > temp) {
                arr[j] = arr[j - gap];
                j -= gap;
            }
            arr[j] = temp;
        }
    }
}
```

Console Output

```
shell sort
unsorted: 23 68 23 31 95 60 7 24 62 88
sorted: 7 23 23 24 31 60 62 68 88 95
```

Observation

- Shell Sort works like an upgraded version of insertion sort and it works on the larger datasets and now by comparing items that are far apart first. It uses a shrinking gap to push smaller numbers toward the front much faster than checking neighbors one by one. Over multiple passes, the gap shrinks to 1, leaving us with a fully sorted list.

Table 8-3. Merge Sort Algorithm

Source Code

```
// Merge Sort
arr = baseArr;
std::cout << "\nmerge sort\n";
std::cout << "unsorted: ";
printArray(arr);
mergeSort([&]arr, left: 0, right: arr.size() - 1);
std::cout << "sorted: ";
printArray(arr);
```

```
// Merge Sort                                                    ⚠ 4 ^
void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort([&]arr, left, right: mid);
    mergeSort([&]arr, left: mid + 1, right);

    std::vector<int> L(first: arr.begin() + left, last: arr.begin() + mid + 1);
    std::vector<int> R(first: arr.begin() + mid + 1, last: arr.begin() + right + 1);

    int i = 0, j = 0, k = left;
    while (i < L.size() && j < R.size()) {
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    }
    while (i < L.size()) arr[k++] = L[i++];
    while (j < R.size()) arr[k++] = R[j++];
}
```

Console Output

```
merge sort
unsorted: 23 68 23 31 95 60 7 24 62 88
sorted: 7 23 23 24 31 60 62 68 88 95
```

Observation

- Merge Sort uses a divide-and-conquer strategy by continually cutting the unsorted array in half. Once the pieces are broken down to single elements, it sorts and zips them back together in order. It handles duplicate numbers) smoothly and outputs the correctly sorted sequence (7 23 23 24 31 60 62 68 88 95)

Table 8-4. Quick Sort Algorithm

Source Code

```
// Quick Sort
arr = baseArr;
std::cout << "\nquick sort\n";
std::cout << "unsorted: ";
printArray(arr);
quickSort( [&] arr,  low: 0,  high: arr.size() - 1);
std::cout << "sorted: ";
printArray(arr);
```

```cpp
// Quick Sort
void quickSort(std::vector<int>& arr, int low, int high) {
    if (low >= high) return;
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) std::swap( a: [&] arr[++i], b: [&] arr[j]);
    }
    std::swap( a: [&] arr[i + 1], b: [&] arr[high]);
    int pi = i + 1;

    quickSort( [&] arr, low, high: pi - 1);
    quickSort( [&] arr, low: pi + 1, high);
}
```

Console Output

```
quick sort
unsorted: 23 68 23 31 95 60 7 24 62 88
sorted: 7 23 23 24 31 60 62 68 88 95
```

Observation

- Quick Sort works by picking a reference point called a pivot, in this case, I chose the last element. It shifts everything smaller than the pivot to the left side and everything larger to the right. It repeats this partitioning on the smaller sub-sections in place until the entire array is fully ordered.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- Yes, we can sort the sub-lists using other algorithms. In this implementation, I used a hybrid combination of 2 sorts which are the quick sort and selection sort, first is the quick sort's partition method splits the array around a pivot element into a left sub-list and a right sub-list and then, selection sort is applied to independently sort both sub-lists, resulting in a fully ordered array.

Source Code

```cpp
int main() {
    srand( Seed: time(0));
    std::vector<int> baseArr( n: 10);
    for (int& num : baseArr) num = rand() % 100 + 1;

    std::vector<int> arr  = baseArr;

    std::cout << "Original Array: ";
    printArray(arr);

    int pivotIndex = partition( [&] arr,  low: 0,  high: arr.size() - 1);
    std::cout << "\nAfter Partition (Pivot is " << arr[pivotIndex] << "):\n";
    printArray(arr);

    selectionSort( [&] arr,  start: 0,  end: pivotIndex - 1);
    selectionSort( [&] arr,  start: pivotIndex + 1,  end: arr.size() - 1);
    std::cout << "\nFinal Sorted Array: ";
    printArray(arr);

    return 0;
}
```

Console Output

```
Original Array: 60 22 19 86 34 61 60 61 49 31

After Partition (Pivot is 31):
22 19 31 86 34 61 60 61 49 60

Final Sorted Array: 19 22 31 34 49 60 60 61 61 86
```

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- Quick Sort will give the fastest time performance because it works by picking a pivot and partitioning the array around it. This lets it organize the numbers right inside the same array, saving time by avoiding extra memory steps.
- Merge Sort and Quick Sort both have O(N log N) time complexity because they use a divide-and-conquer approach. Dividing the array in half repeatedly takes log N steps. At each step, processing all N numbers takes linear time. Multiplying the N numbers by the log N steps gives a total time of O(N log N).

Conclusions

Summary of Lessons Learned

In this lab, I learned how advanced sorting algorithms like Shell Sort, Merge Sort, and Quick Sort work under the hood. I gained a better understanding of how divide-and-conquer strategies break down large datasets to make sorting faster compared to simple algorithms.

Analysis of the Procedure

The procedure involved implementing and observing the behavior of Shell Sort, Merge Sort, and Quick Sort through step-by-step trace tables. Tracking each pass made it clear how Shell Sort uses gaps to pre-sort elements, how Merge Sort splits lists before merging them in order, and how Quick Sort uses a pivot element to partition numbers.

Analysis of the Supplementary Activity

In the supplementary activity, I built a hybrid sorting program that uses the quick sort's partitioning method with selection sort in it. This demonstrated how different sorting algorithms can be paired together to solve specific problems using the quick sort to quickly split the array around a pivot and selection sort to handle the smaller sub-lists.

Concluding Statement / Feedback

I think I did decent in this activity, but I struggled a little bit along the way, especially when tracing the step-by-step partition steps for quick sort. Despite these challenges, I was able to complete all the trace tables and build a working hybrid algorithm for the supplementary task.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.